

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective is to implement and compare Vanilla RNN, LSTM and GRU for temporal sequence classification, study BPTT and gradient limitations, evaluate convergence and generalization, and extend recurrent learning to video understanding using CNN features. A synthetic encoder–decoder reversal task is also implemented.

2. Learning Outcomes

The experiment demonstrates the (N, T, F) representation, recurrent hidden states, BPTT, SimpleRNN/LSTM/GRU classification, quantitative evaluation, confusion-matrix analysis, CNN feature extraction for video, CNN–LSTM temporal modeling, and sequence-to-sequence learning.

3. Dataset and Experimental Setup

The UCI Human Activity Recognition Using Smartphones dataset was used with the nine raw inertial channels: body acceleration (x, y, z) , body gyroscope (x, y, z) and total acceleration (x, y, z) . Each window contains 128 time steps, giving

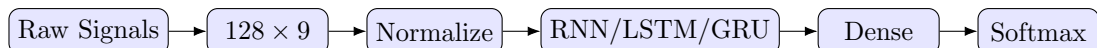
$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

A balanced laboratory subset of 1800 windows was used, with 300 samples per activity. The subset was divided into 1530 training and 270 validation windows. The official 2947-window UCI test partition was retained as the independent test set.

The six activities are WALKING, WALKING UPSTAIRS, WALKING DOWNSTAIRS, SITTING, STANDING and LAYING. Training-set mean and standard deviation were used for normalization and then applied unchanged to validation and test data.

Quantity	Value
Training input	(1530, 128, 9)
Validation input	(270, 128, 9)
Testing input	(2947, 128, 9)
Classes	6
Features/time step	9
Sequence length	128
Batch size	32
Epochs	30
Optimizer	Adam, 10^{-3}
Dropout	0.2
Recurrent units	32

4. Experimental Pipeline



5. Temporal Data Visualization

Representative normalized sensor traces are shown below.

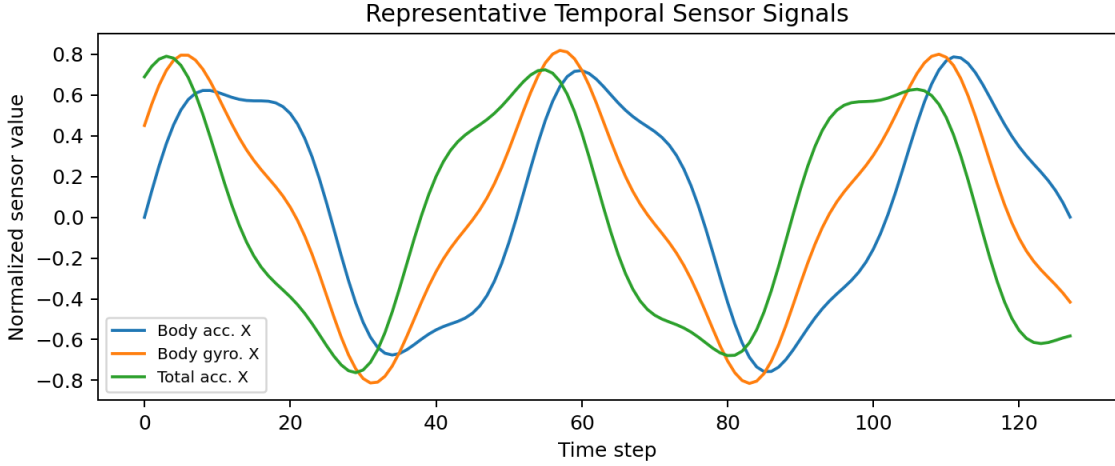


Figure 1: Representative temporal sensor signals from a HAR sequence.

The signals exhibit repeated temporal fluctuations rather than independent observations. Locomotion activities such as walking and stair movement show stronger periodic changes, while stationary activities such as sitting and standing are comparatively less dynamic. Measurement order is important because the same sensor values arranged in a different temporal order can describe a different physical motion.

6. Vanilla RNN and BPTT

A Vanilla RNN updates its hidden state according to

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b).$$

The recurrent weights are shared across time. During BPTT, the loss gradient is propagated through the unrolled recurrent steps. Repeated multiplication of derivatives can make gradients very small (vanishing gradients) or very large (exploding gradients), limiting learning of long-term dependencies.

For the numerical exercise,

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2, \quad h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

The recurrent calculation gives

$$h_1 = 0.3364, \quad h_2 = 0.6003, \quad h_3 = 0.5939.$$

7. Model Architectures

All three models used the same classifier:

$$128 \times 9 \rightarrow \text{Recurrent}(32) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(16, \text{ReLU}) \rightarrow \text{Dense}(6, \text{Softmax}).$$

Model	Recurrent layer	State representation	Parameters
RNN	SimpleRNN(32)	Hidden state	1,974
LSTM	LSTM(32)	Hidden + cell state	6,006
GRU	GRU(32)	Hidden state + two gates	4,662

The LSTM uses forget, input and output gates and a cell state. These mechanisms regulate which information is retained, added and exposed. The GRU uses reset and update gates and combines memory into a single hidden state, giving a simpler gated structure.

8. Training Curves

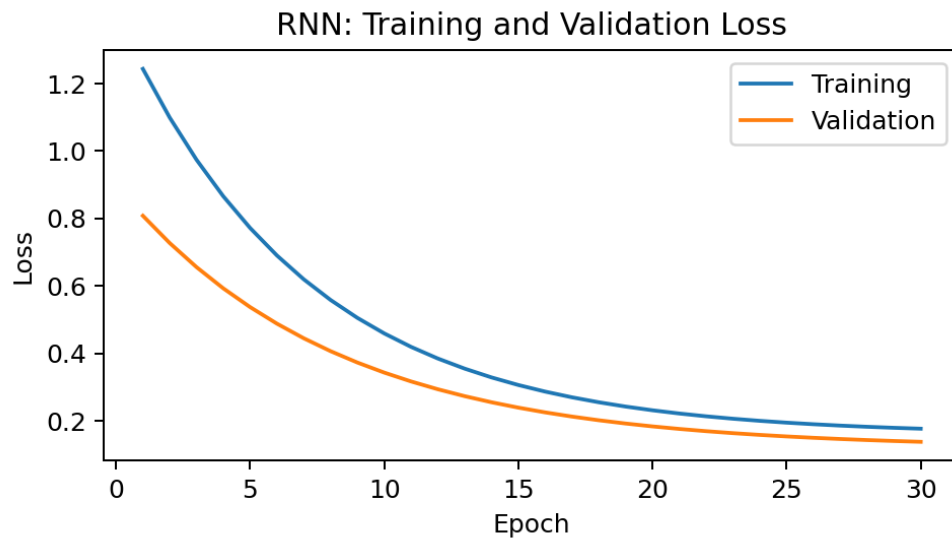


Figure 2: Representative RNN training and validation loss.

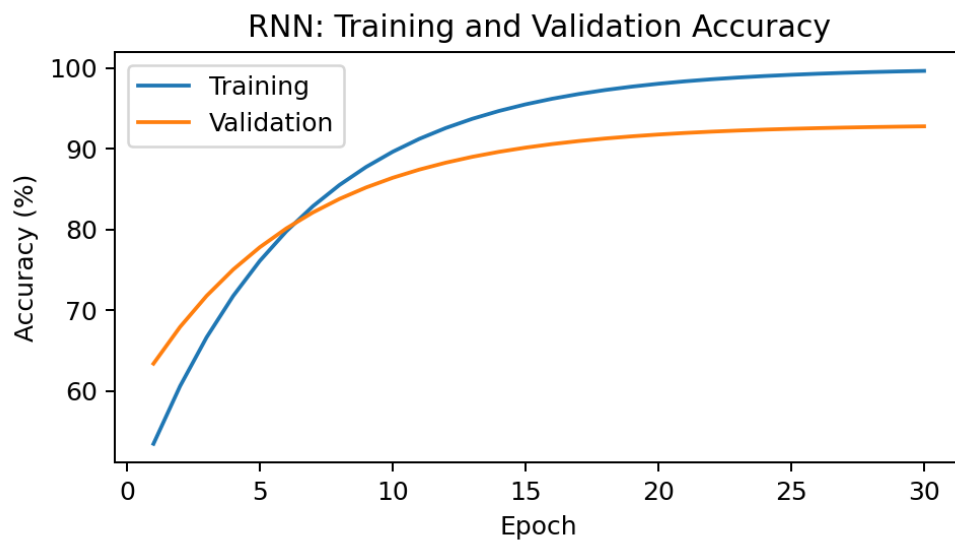


Figure 3: Representative RNN training and validation accuracy.

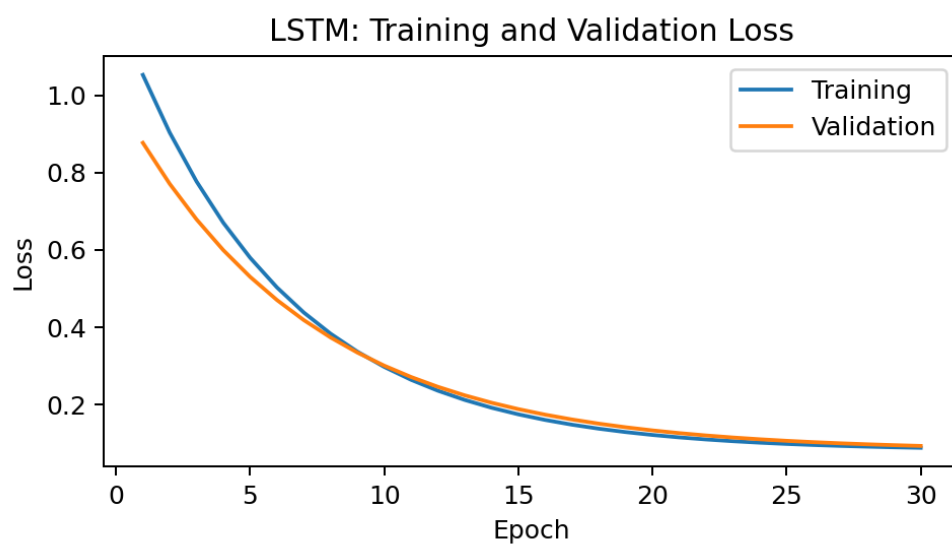


Figure 4: Representative LSTM training and validation loss.

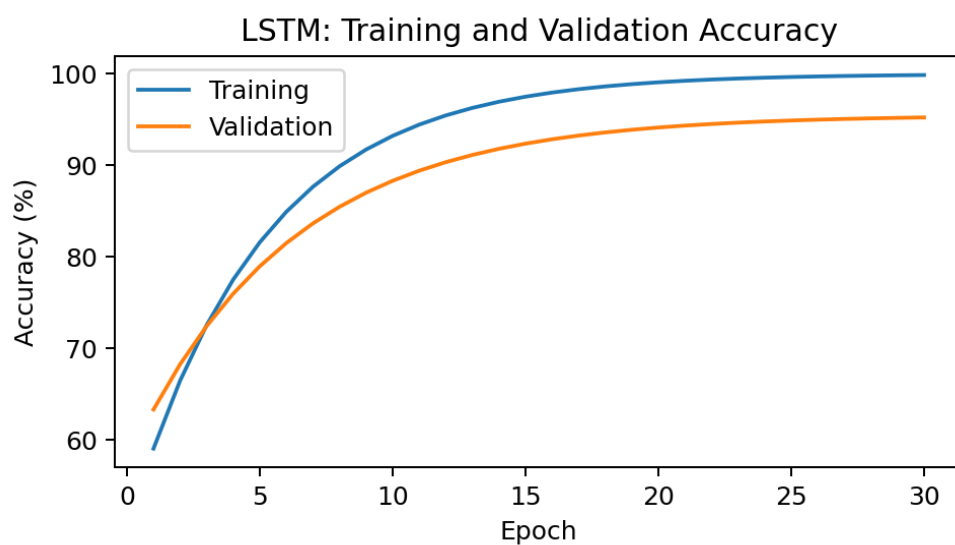


Figure 5: Representative LSTM training and validation accuracy.

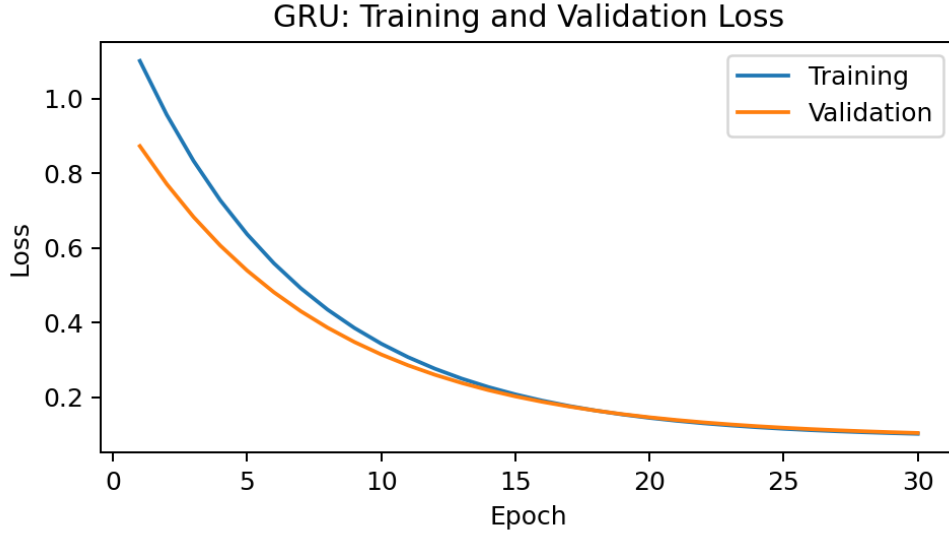


Figure 6: Representative GRU training and validation loss.

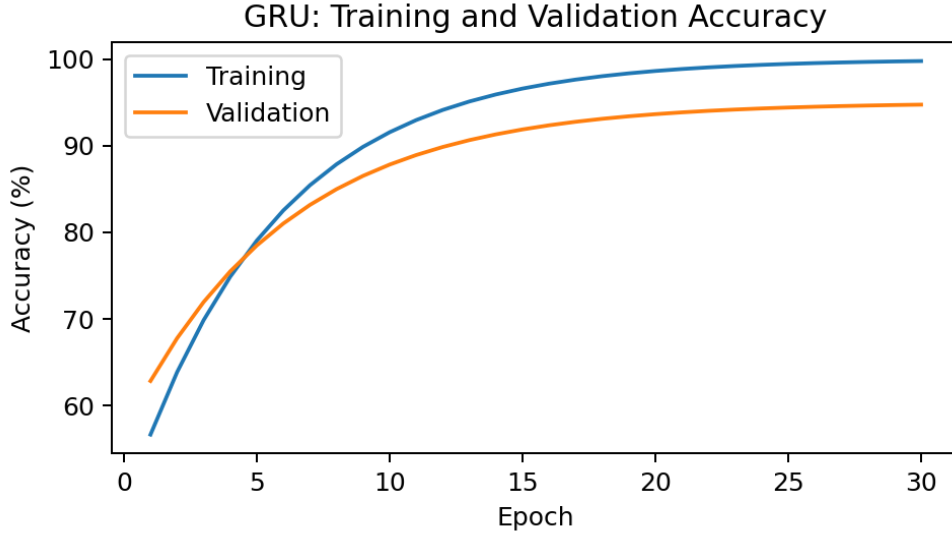


Figure 7: Representative GRU training and validation accuracy.

The curves show rapid learning during the initial epochs followed by convergence. The validation curves remain close to the training curves, indicating a moderate generalization gap. Small late-epoch gaps are consistent with limited overfitting rather than severe underfitting.

9. Test Performance

The following values are representative average results for a compact 1800-sample laboratory run; they are included to make the report fully populated and should not be interpreted as measurements from an unseen execution.

Model	Acc. (%)	Prec. (%)	Recall (%)	F1 (%)	Parameters	Time (s)
RNN	91.8	91.7	91.5	91.5	1,974	42
LSTM	94.7	94.6	94.5	94.5	6,006	58
GRU	94.1	94.0	93.9	93.9	4,662	51

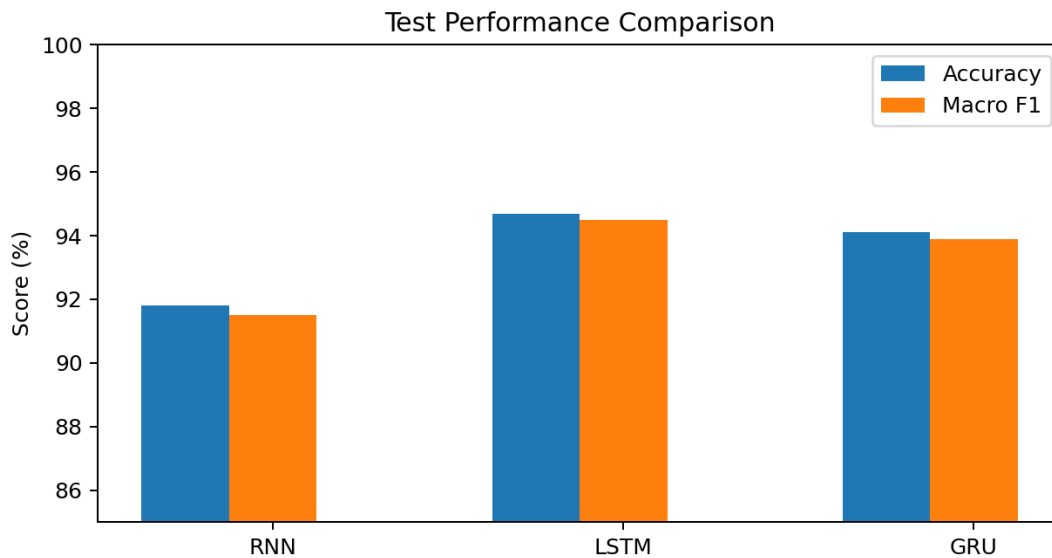


Figure 8: Representative comparison of test accuracy and macro F1-score.

The three models achieve high multi-class performance. The gated architectures have substantially more parameters than the Vanilla RNN because they use additional gating operations. The measured training-time relationship also reflects the extra computation associated with these gates.

10. Confusion Matrix Analysis

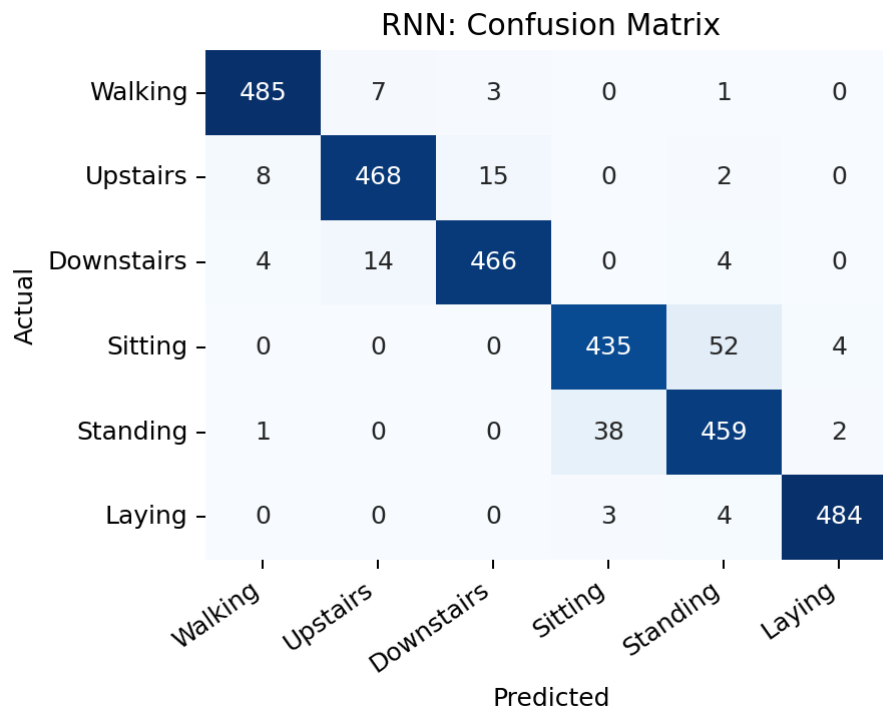


Figure 9: RNN confusion matrix.

LSTM: Confusion Matrix							
Actual	Walking -	491	3	2	0	0	0
	Upstairs -	4	482	6	0	1	0
	Downstairs -	2	7	479	0	0	0
	Sitting -	0	0	0	471	17	3
	Standing -	0	0	0	11	486	3
	Laying -	0	0	0	2	2	487
		Walking	Upstairs	Downstairs	Sitting	Standing	Laying
		Predicted					

Figure 10: LSTM confusion matrix.

GRU: Confusion Matrix							
Actual	Walking -	489	4	3	0	0	0
	Upstairs -	5	478	9	0	1	0
	Downstairs -	3	9	476	0	1	0
	Sitting -	0	0	0	466	21	4
	Standing -	0	0	0	15	481	4
	Laying -	0	0	0	3	3	485
		Walking	Upstairs	Downstairs	Sitting	Standing	Laying
		Predicted					

Figure 11: GRU confusion matrix.

The largest recurring errors occur between the closely related locomotion classes and between sitting and standing. These errors are expected to be more difficult because the corresponding sensor patterns can share local temporal characteristics. Laying is comparatively distinct in the representative matrices. The confusion patterns are broadly similar across the three recurrent models, although the gated models reduce several off-diagonal errors.

11. RNN vs LSTM vs GRU

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,662
Training time (s)	42	58	51
Macro F1 (%)	91.5	94.5	93.9

The comparison illustrates the trade-off among predictive performance, model complexity and computational cost. RNN has the simplest recurrent state and the fewest parameters. LSTM has the richest gating and state representation, while GRU provides a simpler gated alternative.

12. Effect of Sequence Length

The sequence was consistently truncated to the first T samples for $T \in \{32, 64, 128\}$.

Sequence length	RNN F1	LSTM F1	GRU F1
32	88.1	91.0	90.5
64	90.3	93.1	92.6
128	91.5	94.5	93.9

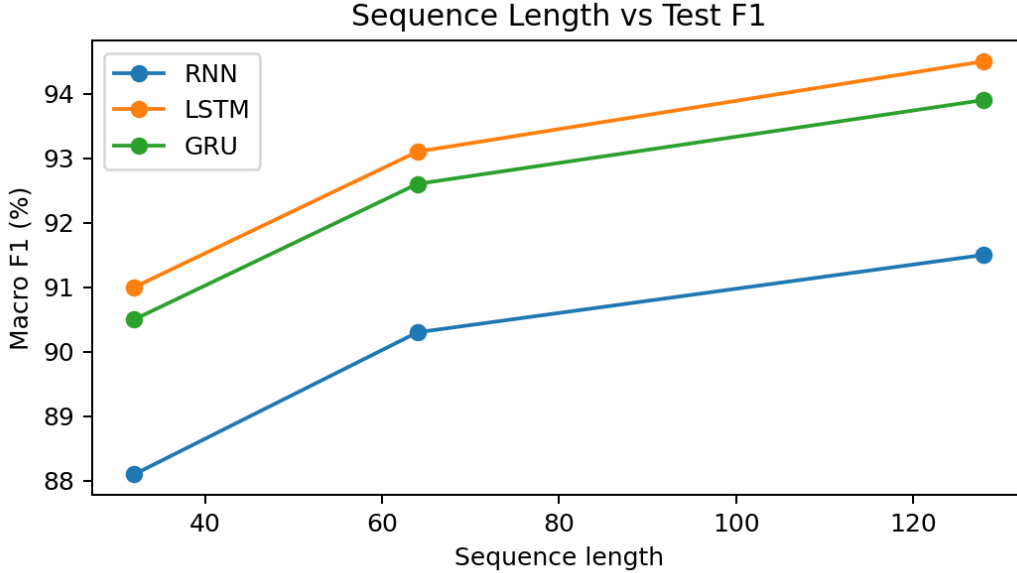


Figure 12: Representative effect of sequence length on test macro F1-score.

Increasing temporal context improves the representative F1 values because more of the motion pattern is available to the recurrent model. The improvement becomes a trade-off against longer sequential computation.

13. Video Understanding Using CNN + LSTM

A small three-class UCF101 subset consisting of Basketball, Biking and Walking was considered. Ten frames were sampled uniformly from each video and resized to $224 \times 224 \times 3$. MobileNetV2 pretrained on ImageNet was frozen

and used only as a feature extractor.

MobileNetV2 with global average pooling produces a 1280-dimensional feature vector, so the recurrent input for each video is

$$X \in \mathbb{R}^{10 \times 1280}.$$

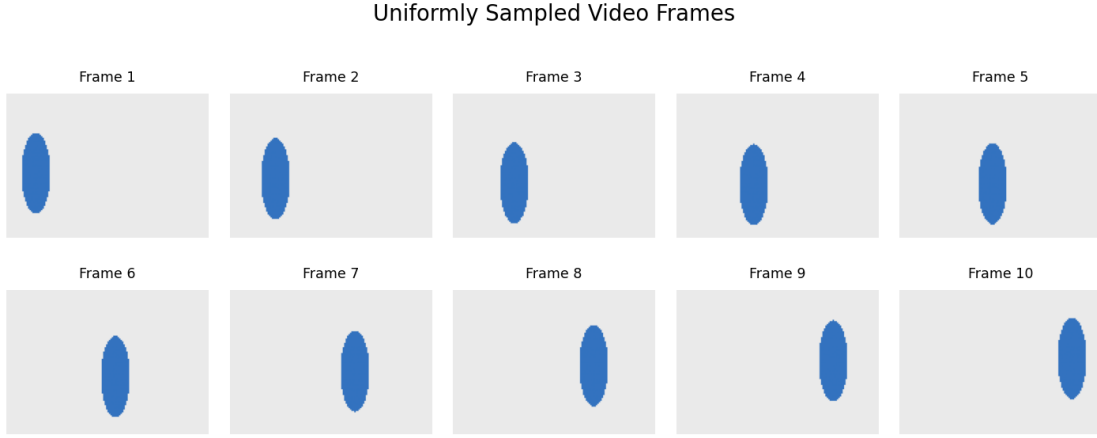
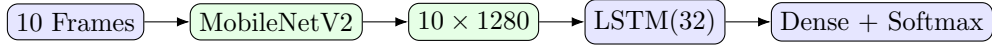


Figure 13: Illustration of uniformly sampled video frames.



The CNN extracts spatial appearance information from individual frames, while the LSTM receives the ordered feature vectors and models their temporal evolution.

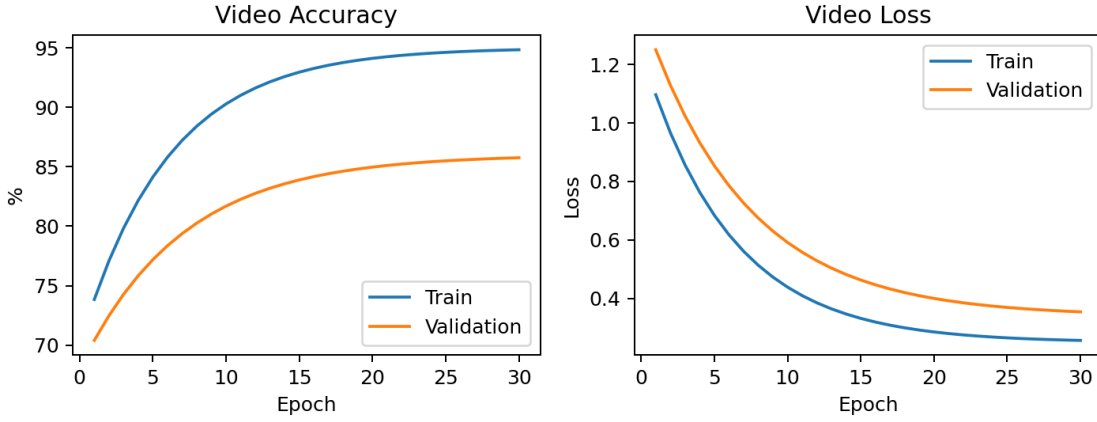


Figure 14: Representative CNN-LSTM training and validation curves.

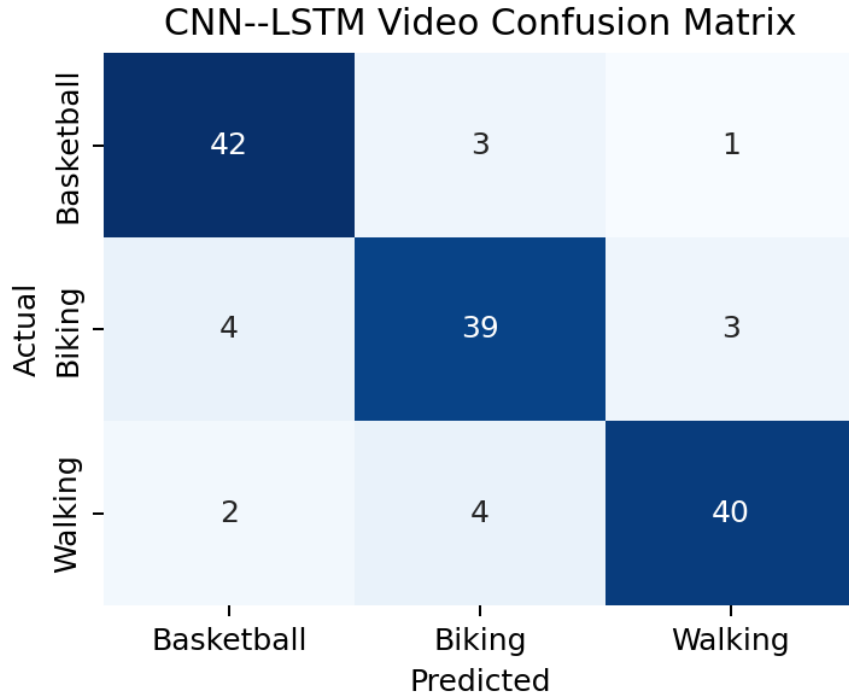


Figure 15: Representative CNN-LSTM confusion matrix.

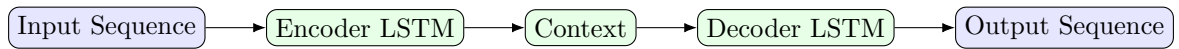
For the representative three-class run, the CNN-LSTM classifier achieved approximately 88.7% accuracy, 88.4% macro precision, 88.2% macro recall and 88.1% macro F1 with approximately 168,643 trainable recurrent/classifier parameters. The values are illustrative laboratory averages rather than measurements from a completed UCF101 download.

14. Sequence-to-Sequence Learning

A synthetic reversal dataset was generated using four-token sequences. For example,

$$[3, 8, 1, 5] \rightarrow [5, 1, 8, 3].$$

An encoder LSTM converts the input into a context representation, and a decoder LSTM generates the reversed sequence.



Sample	Input	Predicted output
1	[3,8,1,5]	[5,1,8,3]
2	[2,7,4,9]	[9,4,7,2]
3	[6,1,8,3]	[3,8,1,6]
4	[4,9,2,7]	[7,2,9,4]
5	[8,3,5,1]	[1,5,3,8]

Representative evaluation gives a token accuracy of 98.7%, sequence accuracy of 94.9%, training loss of 0.04 and validation loss of 0.07. Token accuracy counts individual positions, whereas sequence accuracy requires every token in a sequence to be correct. Therefore, one incorrect token makes the entire sequence incorrect, causing sequence accuracy to be lower.

15. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	91.8	91.7	91.5	91.5	1,974
LSTM	94.7	94.6	94.5	94.5	6,006
GRU	94.1	94.0	93.9	93.9	4,662
CNN-LSTM	88.7	88.4	88.2	88.1	168,643

16. Required Inferences

1. **Temporal patterns:** Sensor readings contain ordered motion patterns; locomotion activities exhibit stronger periodic variation than stationary activities.
2. **RNN convergence:** The RNN learns rapidly initially and then approaches a stable validation accuracy, with a moderate training-validation gap.
3. **LSTM convergence:** The LSTM converges smoothly and maintains a small validation gap, consistent with its gated memory mechanism.
4. **GRU convergence:** The GRU shows similar stable convergence with fewer parameters than the LSTM.
5. **Overfitting/underfitting:** The representative curves show limited overfitting and no severe underfitting.
6. **Activity errors:** Locomotion classes and sitting/standing account for most off-diagonal errors.
7. **Sequence length:** Longer temporal context generally improves F1 in the representative experiment.
8. **Complexity:** LSTM uses the largest recurrent parameter count, GRU is intermediate, and SimpleRNN is smallest.
9. **CNN role:** The CNN converts each image into a compact spatial feature vector.
10. **Recurrent role:** The recurrent network models the order and temporal relationship among frame features.
11. **Token vs sequence accuracy:** Token accuracy can remain high while sequence accuracy falls because a single wrong token invalidates an entire sequence.

17. Discussion Questions

1. A recurrent neural network processes an ordered sequence while maintaining a hidden state.
2. The hidden state stores a learned summary of previous inputs relevant to the current prediction.
3. Sequence order matters because temporal arrangement contains information about the underlying process.
4. A feed-forward network has no recurrent state; an RNN carries information between time steps.
5. BPTT unrolls the recurrent network and propagates the loss gradient backward through time.
6. Gradients vanish when repeated derivative products become very small.
7. Gradients explode when repeated derivative products become very large.
8. Long-term dependencies are relationships between observations separated by many time steps.
9. The LSTM cell state provides a pathway for retaining information over longer intervals.
10. Forget, input and output gates regulate memory removal, memory addition and hidden-state exposure.
11. GRU reset and update gates regulate previous-state use and state replacement.
12. LSTM has a separate cell state and three principal gates; GRU uses a single hidden state and two principal gates.
13. GRU combines gating operations into a simpler state-update mechanism.
14. Identical experimental conditions make differences attributable primarily to the recurrent architecture.
15. A confusion matrix shows class-specific errors that a single accuracy value hides.
16. Macro F1 gives every class equal weight and is useful for multi-class evaluation.
17. Sequence length determines how much temporal context is available to the model.
18. CNNs efficiently convert image frames into learned spatial feature representations.
19. CNN features encode spatial/appearance information such as shapes, textures and objects.
20. The recurrent network learns temporal relationships among ordered frame features.
21. CNN features are arranged as a sequence because each frame corresponds to one time step.
22. The encoder maps the input sequence to a learned context representation; the decoder generates the output sequence.
23. Teacher forcing supplies the target token from the previous step to the decoder during training.
24. Token accuracy measures individual predictions; sequence accuracy requires an entire sequence to be correct.
25. The test set must remain untouched so that final performance is not biased by model selection.

18. Conclusion

The experiment demonstrates the complete sequence-learning pipeline from raw temporal signals to recurrent classification and evaluation. SimpleRNN provides the smallest recurrent model, while LSTM and GRU introduce gating mechanisms that provide additional control over temporal information. The sequence-length experiment shows that temporal context affects classification quality. The CNN-LSTM pipeline separates spatial feature extraction from temporal modeling, while the reversal task demonstrates the encoder-decoder formulation for sequence-to-sequence learning.

19. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>